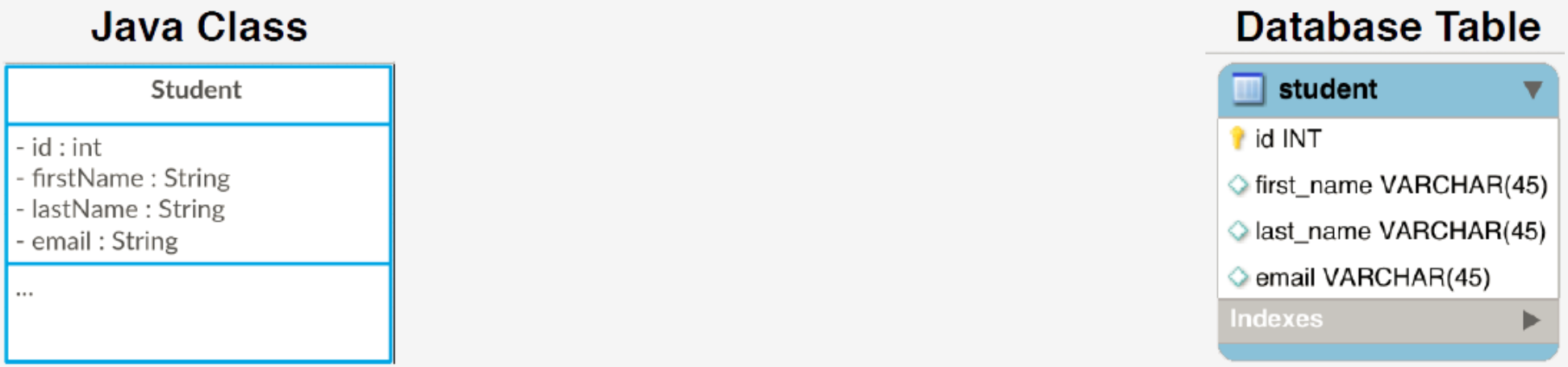
Hibernate

Desarrollo Entorno Servidor

JPA

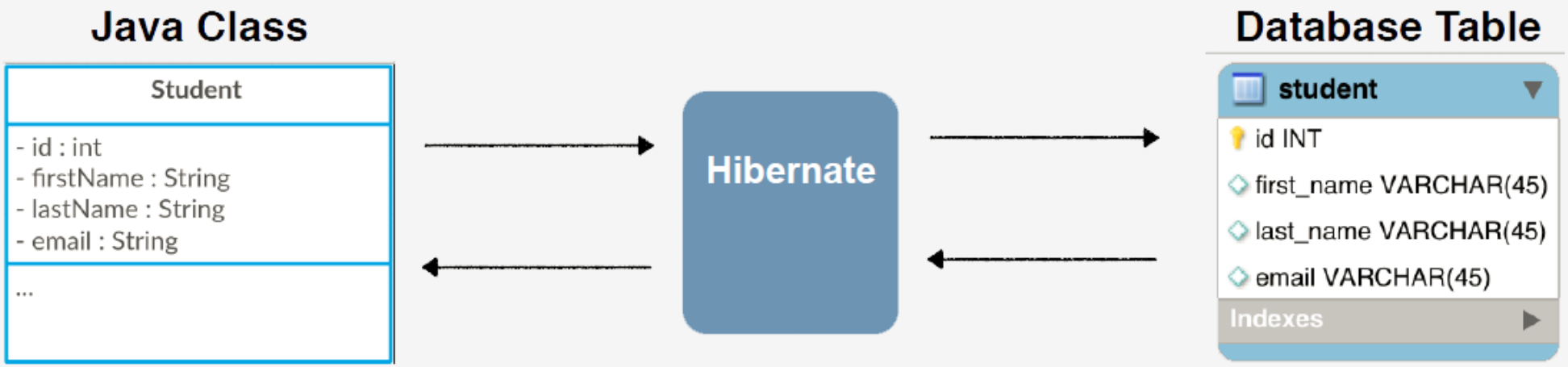
Las bases de datos relacionales almacenan la información mediante tablas, filas, y columnas, de manera que para almacenar un objeto hay que realizar una correlación entre el sistema orientado a objetos de Java y el sistema relacional de nuestra base de datos.



JPA

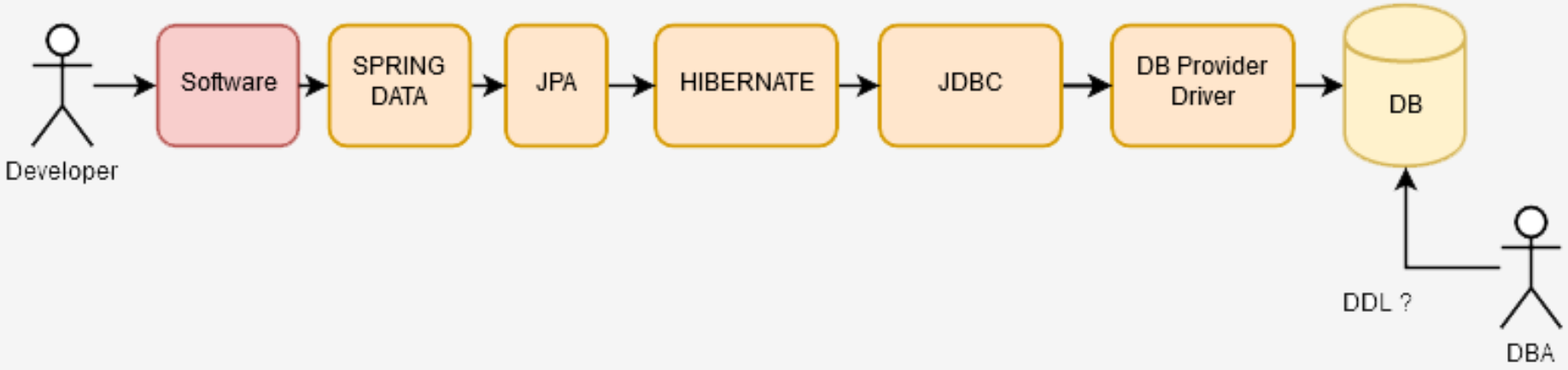
JPA (**Java Persistence API**, API de Persistencia en Java) es una abstracción sobre JDBC que nos permite realizar dicha correlación de forma sencilla, realizando por nosotros toda la **conversión entre nuestros objetos y las tablas de una base de datos.**

Esta conversión se llama **ORM** (**Object Relational Mapping** - Mapeo Relacional de Objetos), y puede configurarse a través de metadatos (mediante xml o anotaciones). **Estos objetos los llamaremos desde ahora entidades (entities).**



Hibernate

JPA establece una interface común que es implementada por un proveedor de persistencia de nuestra elección (como **Hibernate**, Eclipse Link, etc), de manera que podemos elegir en cualquier momento el proveedor que más se adecue a nuestras necesidades. **Así, es el proveedor quién realiza el trabajo, pero siempre funcionando bajo la API de JPA**.

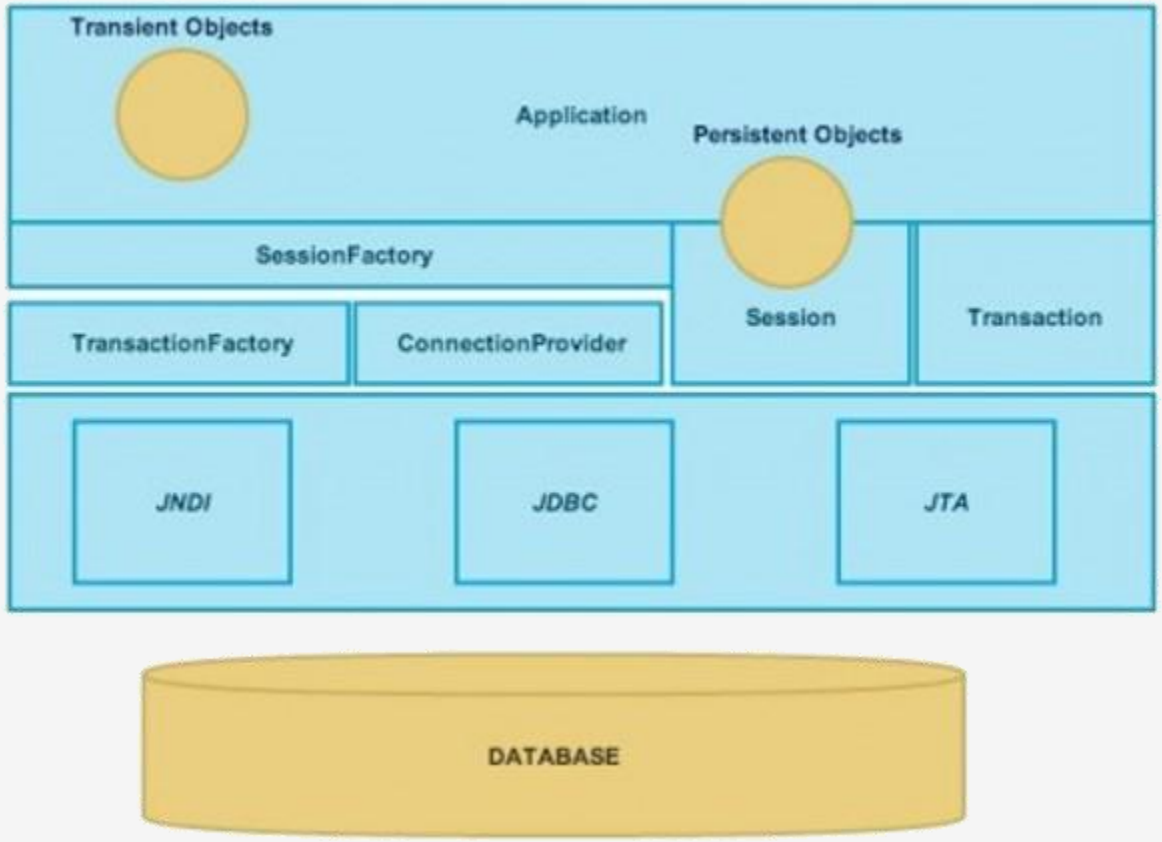


Hibernate

En definitiva **Hibernate** es un framework ORM para Java, proporciona:

- Mapeo de clases Java a tablas y viceversa
- Gestiona transacciones y garantiza consistencia de los datos
- Gestiona los recursos de BD de forma eficiente
- Proporciona una capa de abstracción entre la aplicación y la BD
- Mapeos para joins, colecciones, herencia…
- HQL lenguaje similar a SQL pero orientado a objetos: soporta herencia, polimorfismo y asociación

Hibernate - Arquitectura



Hibernate - Arquitectura

- **SessionFactory (org.hibernate.SessionFactory):** Factoría para crear sesiones para una BD
- **Session (org.hibernate.Session):** objeto envoltorio para una conexión JDBC. Es el objeto que permite a la aplicación comunicarse con la BD
- **Persistent objects**: POJOs o Java Beans que se hacen persistentes o viceversa en una sesión
- **Transient objects**: objetos instanciados en la aplicación pero no asociados a una sesión
- **Transaction (org.hibernate.Transaction)**: Gestiona transacciones aislando de JDBC y JTA
- **ConnectionProvider (org.hibernate.connection.ConnectionProvider)**: Factoria para conexiones JDBC. Gestiona DataSource y DriverManager de forma transparente
- **TransactionFactory (org.hibernate.TransactionFactory)**: Factoria para instancias de transacción

Ciclo de vida: Hibernate

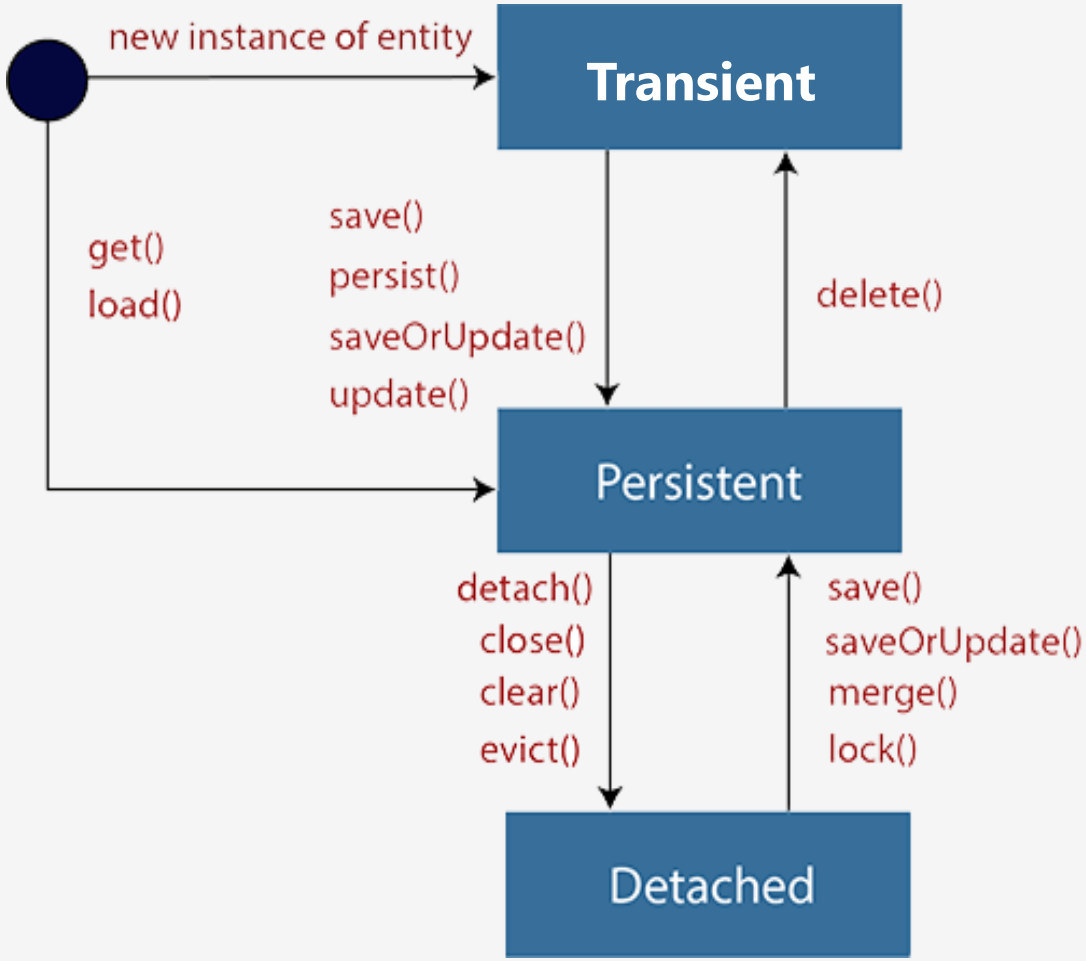
Como hemos mencionado anteriormente, Hibernate mapea los objetos y sus atributos a las tablas y sus columnas.

De este modo, la creación de entidades y los cambios que realizados sobre ellas serán persistidos a la base de datos.

Hibernate determina una ciclo de vida para las instancias de las entidades, en la que considera 3 estados:

- **Transient**
- **Persistent**
- **Detached**

Ciclo de vida: Hibernate.-Transient



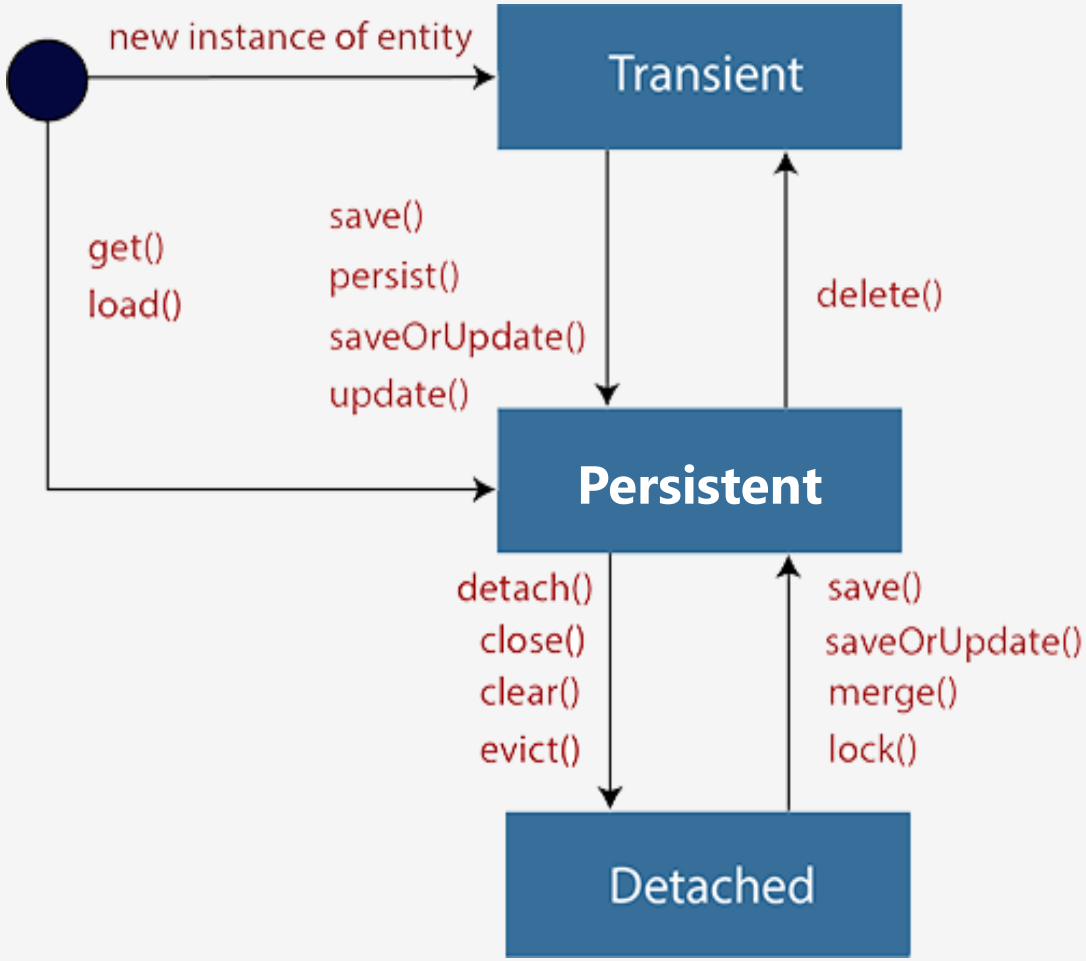
Transient: Un objeto que ha sido instanciado usando el operado **new** y no esta asociado a una sesión de Hibernate. **No tiene representación persistente en la base de datos y no se le ha asignado un identificador.**

```
String nombre = request.getParameter("nombre");
String apellidos = request.getParameter("apellidos");
String username = request.getParameter("nombreUsuario");
String direccionemail = request.getParameter("email");

Profesor p = new Profesor();
p.setNombreProfesor(nombre);
p.setApellidosProfesor(apellidos);
p.setUsername(username);
```

El objeto Profesor ha sido creado pero no tiene un id.
Este objeto existe en nuestro código java pero no en la BD

Ciclo de vida: Hibernate.-Persistent



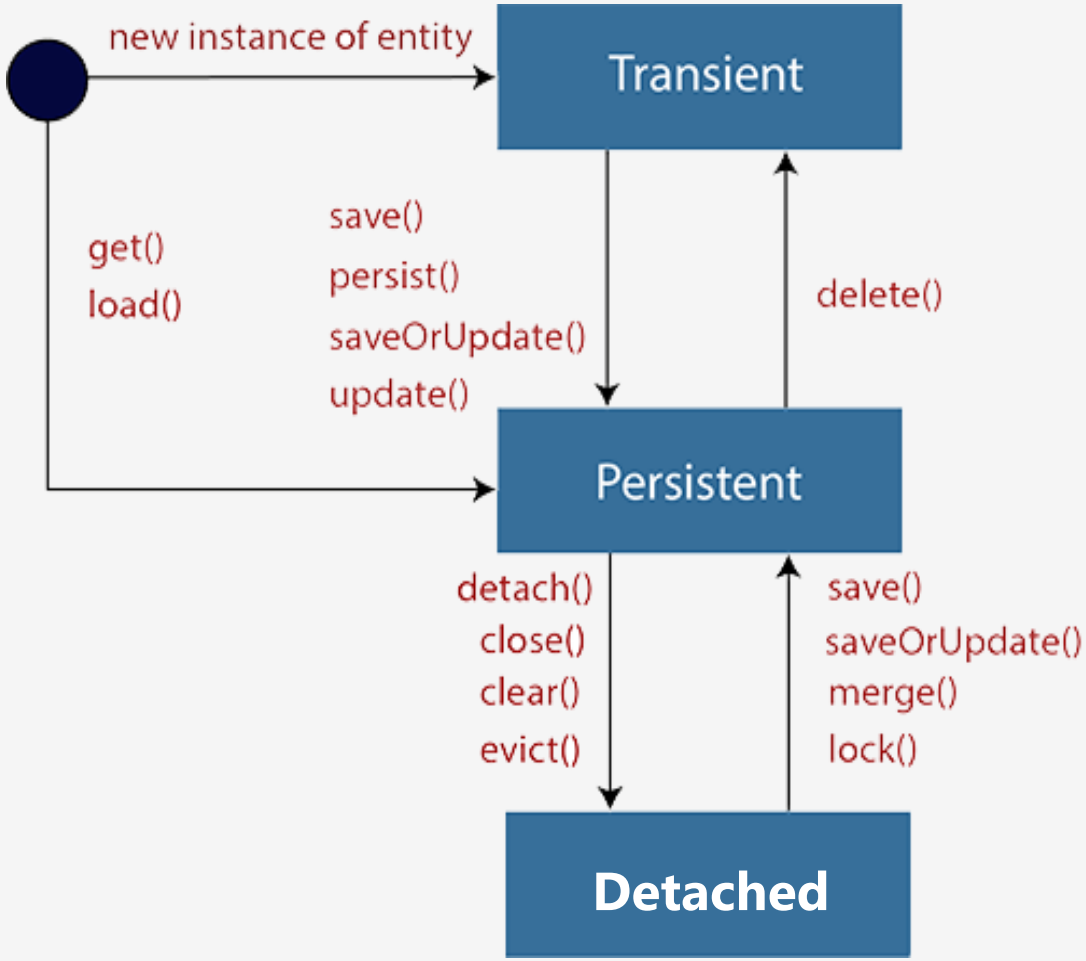
Persistent: un objeto persisitente es aquel que posee un identificador y su representación es igual al de la base de datos. Este objeto puede haber sido guardado (save) o cargado (load)

```
String nombre = request.getParameter("nombre");
String apellidos = request.getParameter("apellidos");
String username = request.getParameter("nombreUsuario");
String direccionemail = request.getParameter("email");

Profesor p = new Profesor();
p.setNombreProfesor(nombre);
p.setApellidosProfesor(apellidos);
p.setUsername(username);
Profesor prof = profesorService.crearPorfesor(p);

@Override
public Profesor crearPorfesor(final Profesor profesor) {
            this.em.persist(profesor);
return profesor;
}
```

Ciclo de vida: Hibernate.-Detached



Detached: un objeto que se ha hecho persistente, pero su Session ha sido cerrada. La referencia al objeto todavía es válida, por supuesto, y la instancia desvinculada (detached) podría incluso ser modificada en

```
//Con este metodo obtenemos al porfesor, un objeto Persistent
Profesor p = profesorService.obtenerProfesor(idProfesor);

//Modificamos el objeto pasa a un estado Detached
p.setUsername(profesorFormulario.getUsername());
p.setNombreProfesor(profesorFormulario.getNombreProfesor());
p.setApellidosProfesor(profesorFormulario.getApellidosProfesor());

//guardamos los cambios el oibjeto vuevle a ser Persistent
Profesor profesormodificado =profesorService.modificarProfesor(p);
```

Entidades

Una entidad va a ser una simple clase Java que deseamos persistir en la base de datos, lo que conocemos como **POJOS**.

Tiene que cumplir las siguientes características:

- Tener un constructor público sin ningún tipo de argumentos.
- Método Getter y Setter para cada propiedad que queramos.
- La anotación **@Entity** encima de la clase.
- Implementar el interfaz **Serializable**

```
@Entity
@Table(name = "PROFESOR")
public class Profesor implements Serializable {

private static final long serialVersionUID = -8668594760203621162L;
@Id
@GeneratedValue(strategy= GenerationType.IDENTITY)
@Column(name = "ID_PROFESOR")
private Long idProfesor;
```

EntityManagerFactory y EntityManager

Cuando lanzamos nuestra aplicación, Spring crea el **EntitytMangerFactory**, este gestiona nuestra conexión a la base de datos a través del **DataSource**, además comienza a buscar las entidades de la aplicación para almacenarlas en un contenedor.

Cada vez que nuestra Aplicación recibe una petición el **EntityManagerFactory** crea un **EntityManager**.

Cada **EntityManager** tiene un entorno de persistencia asociado (**Persistence Context**).

El **Persistence Context** contiene todas las entidades obtenida de la base de datos o creadas por nosotros. El **Persistence Context** es manejado por nosotros a través del **EntityManager**.

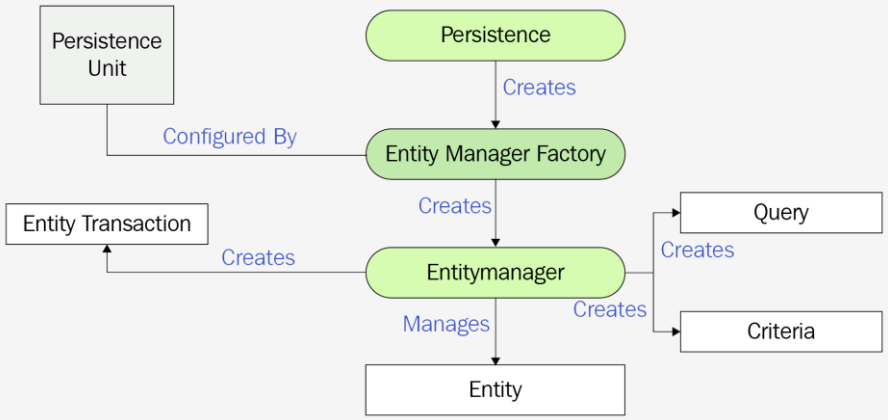
Una vez finalizada la petición el **EntityManger** y **Persistence Context son** eliminados.

EntityManagerFactory y EntityManager

El **EntityManger** es parte de JPA, es el encargado de gestionar el ciclo de vida de las entidades en JPA, además es el responsable del iniciar y cerrar las transacciones de base de datos al igual que de hacer Rollback.

Cuando hacemos uso de un entidad y de alguna operación relacionada a dicha entidad el **EntityManager** será el responsable de iniciar y cerrar la conexión.

El **EntityManager** nos proporciona una serie de operaciones básicas para gestionar nuestras entidades: persist, merge, remove, find…



EntityManger Métodos

- **persist(Object):** Crea la entidad en la base de datos.
- **find(Object):** Obtiene una instancia de la entidad en base de datos en función de la clave primaria
- **merge:** Guarda los cambios sobre una entidad existente en base de datos.
- **createQuery(query):** permite lanzar un query a la base de datos
- **remove():** Eliminar una entidad de la base de datos en función de su clave